

A Model-Driven Approach for Behavior-Driven GUI Testing

Hendrik Bänder
itemis AG
Bonn
buender@itemis.de

Herbert Kuchen
ERCIS, University of Münster
Münster
kuchen@uni-muenster.de

ABSTRACT

Behavior-driven development (BDD) brings requirement specifications and their test cases closer together by using a ubiquitous language to describe requirements that are automatically mapped to test methods. Although industry-proven tools support this automated requirement mapping, the test methods need to be implemented manually. The approach presented in this paper transforms BDD-like requirement specifications and low fidelity graphical user interface (GUI) descriptions in the format of wireframes into automatically executable graphical user interface test cases. The requirement descriptions are defined in a domain-specific language (DSL) that obeys the rules of the ubiquitous language enhanced by references to screens and widgets in a graphical user interface model. The paper reports on a controlled AB/BA crossover experiment comparing the efficiency of our approach to the industry-proven JBehave tooling. The results show that the introduced approach on average takes 63% less time to specify automated tests and thereby diminishes the problem of graphical user interface test cases being expensive to write. The experimental results were supported by experience made applying the approach in three differently sized components at a large Swiss bank.

CCS CONCEPTS

• **Computing methodologies** → **Modeling methodologies**; • **Software and its engineering** → **Domain specific languages**; **Application specific development environments**; • **General and reference** → *Empirical studies*;

KEYWORDS

Domain-specific language, Model-Driven Engineering, GUI Test Case Generation, Behavior-Driven Development

ACM Reference Format:

Hendrik Bänder and Herbert Kuchen. 2019. A Model-Driven Approach for Behavior-Driven GUI Testing. In *The 34th ACM/SIGAPP Symposium on Applied Computing (SAC '19)*, April 8–12, 2019, Limassol, Cyprus. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3297280.3297450>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAC '19, April 8–12, 2019, Limassol, Cyprus

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-5933-7/19/04...\$15.00

<https://doi.org/10.1145/3297280.3297450>

1 INTRODUCTION

The graphical user interface of an application is the main point of interaction for the majority of its users. Therefore, it needs to be tested thoroughly, at best by automated, reproducible test cases. However, automated test cases are often claimed to be expensive to write, time-consuming to execute, and brittle [18]. The presented approach challenges especially the first by integrating low-fidelity prototypes [7] with BDD-like requirement specifications.

Behavior-driven development [8] aims at turning requirements specifications that are defined using a so-called ubiquitous language [10] into executable test cases and tries to keep both close together. Evolving from test-driven development, BDD has been widely adopted in agile projects in research and practice [5]. In the early stage of a project, business analysts, developers, and testers together describe the behavior as well as the expected business value of each feature. The feature definitions are further detailed by scenario descriptions, which tools such as JBehave [15] or Cucumber [35] map automatically to test methods. Finally, these test methods need to be implemented manually to make the requirements specification executable as test cases [35].

Although behavior-driven development brings requirements definitions and test specifications closer together, it does not leverage the full potential of all artifacts that are available in a typical development process. A recent survey has shown that 68% percent of agile projects use low fidelity prototyping [13]. Wireframes [4] provide a rough, conceptual sketch of a graphical user interface. They are a typical format for low fidelity GUI descriptions. More detailed alternatives are UI-mockups or prototypes [7].

The two main shortcomings of the available BDD tools are the lack of integration with GUI descriptions and consequently the manual effort needed to implement test methods. Here, we present a model-driven approach that extends the behavior-driven development method by including low-fidelity prototypes in order to mitigate the problems mentioned above. The introduced specification language Slang integrates BDD-like feature descriptions with wireframe mockups to increase test case creation efficiency by enabling complete generation of automatically executable GUI test cases.

The present paper investigates the potential of combining BDD-like feature description with low-fidelity prototypes in order to fully generate automatically executable test cases. For the purpose of quantifying the economic advantage of the introduced approach, we conducted a controlled experiment. The AB/BA crossover experiment [20] challenges the null-hypothesis claiming that the efficiency (measured in time per test case) for specifying automated graphical user interface test cases is equal to the efficiency of JBehave. Section 5.1 explains the rationale behind choosing JBehave over Cucumber or Squish as tool for comparison. In addition to

the experimental results, the paper reports on first industrial experience collected at a large Swiss bank, where three differently sized components from a practical project used Slang to specify and automate graphical user interface test cases.

Having discussed related work in Section 2, Section 3 elaborates on the capabilities of Slang exemplified by describing features and turning them into test cases automatically for a calendar web application. After describing the technical foundations in Section 4, Section 5 reports on an experiment carried out to analyze the efficiency in terms of times spent of Slang compared to JBehave. Subsequently, Section 6 discusses the experimental results before we conclude in Section 7.

2 RELATED WORK

Behavior-driven development as introduced by Dan North [8] consists of six basic principles. These principles include the ubiquitous language needed to define features and scenarios as well as the automated acceptance testing with mapping rules [5]. However, generating the implementations of the test methods is not supported by any of the reviewed tools so that it remains a manual task.

In contrast, Lazăr et al. introduced an approach integrating BDD and model-driven development to enable the generation of test methods [14]. Based on executable UML enhanced by stereotypes, test methods can be generated and the BDD-like definitions become automatically executable. The focus is on "algorithmic and data-intensive types of programs" [14], while the focus of this research is on information systems. Consequently, the approach introduced by Lazăr et al. does not cover the integration of graphical user interface description. In addition, the concrete textual syntax used to specify scenarios also includes business and verification code leading to complex scenario descriptions. In contrast, the specification language Slang, introduced by this paper, and especially the Feature DSL is completely independent from technical details. Further, it comes with language constructs that read like grammatically correct English sentences, which are suitable also for non-technical domain experts.

Another way to interpret sentences in natural language is natural language processing (NLP) that focuses on turning textual documents written in natural language into formal representations as explained by Collobert and Weston [6]. While the usage of NLP for behavior-driven development as suggested by Soeken et al. [28] has no restrictions on the keywords or grammar constructs used, the ubiquitous language used by BDD tools and frameworks offers a restricted number of keywords to enable the easy interpretation of statements. Consequently an alternative solution is keyword-driven testing that describes test cases by concatenating keywords and parameters [37]. Although the behavior-driven GUI testing approach limits the number of usable keywords, the main goal is to support feature and scenario definitions that read like natural language. Further, the presented approach unifies requirement descriptions and test scripts in a single feature definition file. To summarize, the approach presented in this paper is less flexible and complex than natural language processing, because it limits the number of valid keywords such as "given", "when", "then". Yet, it is more flexible

and readable than keyword-driven testing since test cases can be expressed in human-readable sentences.

The potential benefits of including GUI descriptions such as wireframes or prototypes in a model-driven development process have been elaborated by Rivero [23]. In particular Sanchez et al. [24] have shown that complete graphical user interfaces can be generated from wireframe models. An approach to combine test-driven and model-driven development was introduced by Luna et. al [17]. By deriving application models from test scripts, they leverage the benefits of both approaches such as short-development cycles and less error-prone creation of code.

Although it might be possible to generate the GUI or even the whole application from the requirements and wireframe descriptions, the present paper focuses on generating GUI test cases. There is already a variety of tools available to create automated tests for graphical user interfaces. In particular, Squish is a test automation tool that supports BDD-like test scripts [29]. However, like JBehave it will only generate a method stub that needs to be implemented manually.

Besides the tools mentioned above, there are model-based testing (MBT) approaches. As described by Utting and Legeard [30], MBT focuses on creating an abstract description of the system under test in an early stage. The model is then used as input for fully generating black box system tests. The GUITAR tool introduced by Nguyen et. al [21] is an example of a model-based GUI testing tool. Models are created by a reverse engineering mechanism. These models are the basis for test oracles, system verification, and test case generation for multiple platforms. While MBT focuses on creating a test model for the system, the introduced approach uses model-driven engineering techniques to generate test cases. Following the idea of behavior-driven development, our approach merges the system definition with the test case specification.

3 MATERIAL - BEHAVIOR-DRIVEN GUI TESTING

3.1 Specifying the Calendar Web Application

The capabilities of Slang are demonstrated by a calendar web application including the corresponding wireframe, feature and mapping definitions. After a successful login, the application offers an editable list of appointments. The web application's user interface consists of a login and an overview screen, which are modeled using a sketching tool such as Wireframesketcher [34]. The behavior and the business value of each feature is described in a Slang feature definition file. In order to connect the wireframe model elements, such as screens and widgets, to their implementations, a Slang mapping description file is created. Finally, all three artifacts are processed by a generator that creates automatically executable test scripts.

3.2 Calendar Web Application Screen Definition

Figure 1 shows a wireframe model of the login screen that has two text input fields and one button.

After entering a valid "username" and a valid "password", the login can be executed by clicking the *Login* button. This leads to

Figure 1: Wireframe model describing the Login screen.

the overview screen depicted in Figure 2. The expected behavior of the login screen is described formally within a feature definition file that contains different scenarios.

3.3 Slang Feature Definition

Listing 1 shows the Slang feature specification for the login screen. It is divided into three main parts. The first part defines the *feature name*, which in this case is *Login*. The second part of the feature definition contains the *user story* that specifies the business value of the feature from the user perspective [5]. The third part starting with the keyword *Scenario* describes different scenarios for each feature using grammar constructs that are a subset of the ubiquitous language gherkin [35] enhanced by the functionality to reference wireframe model elements.

```
Feature Login

As a "Calendar User"
I want to "login to the calendar application"
In order to "manage my appointments"

Scenario ValidLogin
  Given I am on the Login screen
  when I type "johnDoe" in the Username textfield
  and I type "Password" in the Password textfield
  and I click the Login button
  then I am on the Overview screen
  and the Appointments table is visible
```

Listing 1: Slang Feature Description: Login Screen.

The scenario in Listing 1 displays the description of a successful login. The *ValidLogin* scenario starts with the context area, which sets a specified screen into focus. In each *command* section, the first command starts with the keyword sequence when I, while all following commands in the same section begin with and I.

The first command types the value "johnDoe" into the Username textfield. The other two commands are concerned with entering the correct password before the Login button is pressed. Based on the context, the possible widgets, actions and their parameters are determined dynamically and proposed to the user by the IDE of the Feature DSL while typing. The references can be validated at any time leading to early detection of scenarios affected by renaming or removing widgets. Consequently, potential errors within a test case due to changing GUI mockups can be determined, before the test case has been executed.

The expected response of the application to the event triggered by clicking the button is described in the *assertion* section, in which the screen content is checked. The keyword sequence then I

Date	Start	End	Topic	Action
2016/04/01	08:00	10:00	Status Meeting	
2016/04/01	12:00	13:00	Lunch	
2016/04/04	09:00	10:00	Team Meeting	

Figure 2: Wireframe model describing the Overview screen.

marks the beginning of the assertion section. The first Slang assertion within the scenario description as shown by Listing 1 asserts whether the screen has been changed to the Overview screen. The next assertion from Listing 1 introduced by the keyword sequence and the is now executed in the context of the Overview screen. Thus, only widgets from the Overview screen are valid in this part of the assertion section.

Figure 2 shows the Overview screen that is shown after a successful login. In order to describe further scenarios for this Overview screen, a separate feature description file needs to be created.

3.4 Slang Mapping Definition

After having created screen and feature descriptions, a *Slang mapping definition* to map wireframe elements to their implementation is required.

```
Mapping Login
  locator {fragment: "#/login"}
  label {"Login"}

Username{
  locator: "#Username"
}
Password{
  locator: "#Password"
}
Login{
  locator: "#Login"
}
```

Listing 2: Slang Mapping Description: Login Screen.

Listing 2 shows the Slang mapping description for the Login screen. The first part of the file states the relative URL *#/login* needed to find the screen within the application. After the keyword *label*, the label of the application window is displayed, used to identify and verify whether the correct screen is shown.

The second part of the Slang mapping file contains one entry for each widget on the screen. As in the Slang feature file, Username, Password, and Login are references to the elements of the wireframe model. For each widget, there is one *locator*, which in case of a web application could be the ID of the element in the HTML document tree [33].

3.5 Generating Automatically Executable Test Cases

Eventually, the Feature and Mapping Generator create automatically executable test scripts and *page objects* [19], respectively. Although Feature and Mapping Generator as shown by Figure 3 rely on information from the wireframesketcher model, they are independent from each other. While the Feature Generator is executed when a Slang Feature file is saved, the Mapping Generator is triggered when a Slang Mapping file is saved.

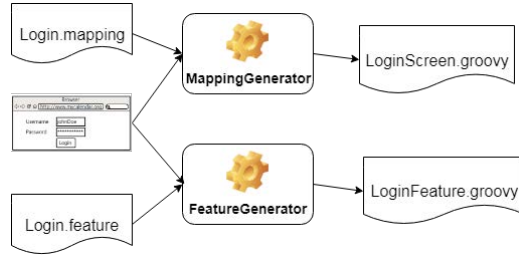


Figure 3: Generation Process.

Figure 3 shows the three files on the left, namely `Login.mapping`, `Login.screen`, and `Login.feature` that are input for Feature and Mapping Generator. The latter combines mapping and screen definition to create a *page object* per screen. The page object encapsulates all interactions, such as clicking, typing, or resolving a label, per screen.

In addition to the page objects, the Slang Feature Generator produces test scripts based on the information from the screen and feature files. Every scenario is turned into a test method containing all corresponding actions and assertions. As shown by Figure 3, the available default generators will create test scripts and page objects in the groovy programming language [12]. Test script and page object will be explained in more detail in Section 4.5.

4 SLANG TECHNICAL FOUNDATIONS

4.1 Architecture

The language workbench to create wireframe, feature and mapping specifications is defined using the Xtext language workbench [3].

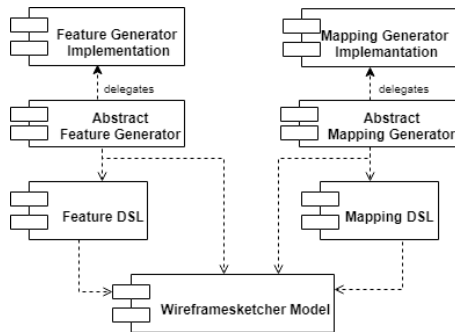


Figure 4: Slang Architecture Overview.

Figure 4 shows the architecture of Slang divided into different components. First, the Wireframesketcher component is located at the bottom of Figure 4. This component contains the graphical editor as well as the underlying formal model which is used to store screens and their widgets. The functionality offered by Wireframesketcher to create and maintain GUI models has been reused completely. However, some customizations to integrate with the Feature and Mapping DSLs as described in Subsection 4.2 have been implemented.

As shown by Figure 4 the domain-specific language parts of Slang, namely *Feature DSL* and *Mapping DSL* are located in separated and independent components. Both encapsulate the grammar definitions and the tooling implementations to support context-sensitive proposals, validations, and syntax highlighting for the Feature and Mapping DSL, respectively. Additionally, Figure 4 shows the *Abstract Feature Generator* and *Abstract Mapping Generator* components that are also independent from each other. While these components implement the integration into the Eclipse IDE in order to start the generation process when a model file has been changed, they delegate the generation to a dedicated component. By decoupling the generator implementation from the DSL definitions, generators can be switched at runtime to generate test cases for different GUI frameworks, such as web or Eclipse Rich Client Platform (RCP) [31] applications, without changing the DSLs.

4.2 Slang Wireframesketcher Integration

The GUI models are created using Wireframesketcher, a tool for creating low-fidelity prototype descriptions for GUI based applications [24]. It is well integrated in the Eclipse tool-chain and stores the GUI models as instances of an Ecore metamodel that is available in an open source repository [34]. In order to use Wireframesketcher models as basis for feature and mapping descriptions, the underlying Ecore model has been customized.

Marker interfaces have been introduced to enable context sensitive proposals within the feature definitions. Besides the existing ones such as `ClickSupport` and `SelectionSupport`, additional concepts such as `DoubleClickSupport` and `BooleanSelectionSupport` have been added. Based on the marker interfaces implemented by a widget, the proposals of available actions within the Slang editor at the current cursor position are determined. Additionally, the Resource Description Strategy [3] of the Wireframesketcher model has been extended to work with nested elements on a screen, such as widgets in containers or widgets in tables. The adjustments described above extend the capabilities of the underlying model, but have no side-effects on the remaining Wireframesketcher tooling for creating sketches.

4.3 Slang Feature DSL

In Listing 3 a slightly simplified grammar of the Slang Feature DSL in Extended Backus-Naur Form (EBNF) is provided¹. The Slang Feature DSL grammar supports all language concepts as specified by the gherkin language which is also supported by JBehave. In addition to the features used in Listing 1, the grammar from Listing 3 contains an `ImplementedCommand` and an `ImplementedAssertion`

¹The postfix operators “?” and “*” indicate optional and arbitrarily repeated features, respectively. “|” divides alternatives. Terminal symbols appear in single quotes.

rule. These rules are a concept of the Feature DSL that can be used to specify custom behavior, e.g., setting or checking the content of a custom widget.

```
Scenario:
  'Scenario' Id
  'Given' 'I' 'am' 'on' QualifiedName 'screen'
  CodeStatement*
CodeStatement:
  ('and' 'I' | 'when' 'I') AbstractCommand |
  ('and' 'the' | 'then' 'the') AbstractAssertion |
  ('then' 'I' 'am' 'on' 'the' QualifiedName 'screen')
AbstractCommand:
  Command | ImplementedCommand
Command:
  Action 'the' WidgetWrapper
Action:
  'click' | ('type' String 'in')
WidgetWrapper:
  QualifiedName ('textfield' | 'button' | 'table' | ...)
AbstractAssertion:
  Assertion | ImplementedAssertion
Assertion:
  WidgetWrapper 'is' ('enabled' | 'visible')
ImplementedCommand:
  String
ImplementedAssertion:
  String
QualifiedName:
  Id ('.' Id)*
```

Listing 3: Feature Grammar in EBNF (simplified; partial, ignoring e.g. rules for identifiers (Id) and strings).

Listing 3 also shows a `QualifiedName` rule used to establish the connection between *Slang feature descriptions* and *wireframe* model elements. Further, it defines a scenario as an arbitrarily mixed sequence of commands and assertions, e.g. for larger processes taking place on a single- or multi-screen application.

4.4 Slang Mapping DSL

The Slang Mapping DSL is required to map the logical elements within a wireframe model such as `Screen` or `Widget` to their implementation. As shown in Listing 2, Slang mapping files are defined per `Screen` and consist of two parts.

```
Mapping:
  'Mapping' QualifiedName Url
  WidgetMapping*
Url:
  'locator' '{ ('fragment' | 'path') ':' String '}'
WidgetMapping:
  QualifiedName '{'
  'locator' ':' Id
  ('driver' ':' widgetDriver=QualifiedName)? '}'
QualifiedName:
  Id ('.' Id)*
```

Listing 4: Mapping Grammar in EBNF (partial).

The first part references the mapped screen by its `QualifiedName`, which establishes the integration between *Slang mapping definition* and *wireframe* model. Additionally, it holds a `locator` attribute containing a relative URL to find the screen within the application under test. In the example used in Listing 2, the `locator` attribute holds a fragment of a URL. For applications where screens are reachable through menus, such as desktop or mobile applications, the `locator` string is introduced by the keyword `path`.

The second part contains the Slang mapping information for all widgets on the particular `Screen`. A `WidgetMapping` consists of a reference to the wireframe model element and the `locator` attribute. The `locator` attribute is an `Id` that identifies the widget within the screen. For a web application, this might be the ID of an element in the DOM tree. In addition, the `WidgetMapping` grammar rule contains an optional `driver` attribute that references a specific implementation for retrieving widgets from the application under test. Specialized drivers can be provided to use additional properties such as `label`, `position`, or `color` for allocating widgets on a screen at runtime. The additional customization possibility is particularly powerful if widgets are dynamically created with an a-priori unknown ID.

4.5 Slang Feature and Mapping Generator

After having defined wireframe, feature and mapping descriptions, the Feature and Mapping generator create all artifacts required to automatically execute GUI test cases. For both generators a default implementation is available that generates test scripts for automatically testing web applications. In addition, there is a proprietary Feature and Mapping Generator for RCP applications using the Standard Widget Toolkit (SWT) [9] as GUI framework. The feature and mapping language register a builder participant in Eclipse that is called when a project is built after saving a Slang feature or mapping file. Subsequently the builder participant calls the specific generator. Based on a reference application as proposed by [32] the generator was implemented with Xtend [36], a Java-like programming language with additional features such as type interference and template expressions. The latter are beneficial for generating source code since they facilitate working with multi-line, highly dynamic strings while maintaining formatting.

```
class LoginScreen extends Page{
  static url = "#/login"
  static at = { title == "Login" }
  static content = {
    username (required: true) { module org.bdd.module.
      TextFieldDriver, $("#username") }
    password (required: true) { module org.bdd.module.
      TextFieldDriver, $("#password") }
    submit (required: true) { module org.bdd.module.ButtonDriver, $(
      "#login") }
  }
}
```

Listing 5: Generated page object.

Listing 5 shows the page object for the Login example that is generated as an instance of the `Page` class. For the login screen the generated code starts with giving the `url`. The generated code continues after the `at` attribute containing a function used to check whether the correct page is displayed. The static attribute `content` defines templates to find widgets on the inspected page. Template definitions consist of a name, additional options, a driver, and a function to find the element. The default driver for allocating widgets on a screen, e.g., `org.bdd.module.TextFieldDriver` for text fields, is used if no specific driver is specified by the mapping file. In the sample shown above, the function `$("#username")`, e.g., returns all elements on the page where the HTML DOM ID equals `"#username"`.

Listing 6 presents the generated test script for the Login example. The test script is generated using the groovy programming language leading to a concise, yet, executable test specification. In addition, it interacts well with the page objects also generated as groovy classes. Listing 6 starts with the given block that opens the login page by using the framework method `to`. The parameter of the `to` method references the Login screen page object. Based on the `url` and `title` properties (see Listing 5) it is verified that the correct screen is shown. Afterwards, the script tries to find the “username” and “password” field within the shown screen by evaluating the page objects content attribute (see Listing 5). Subsequently, the script sets the “username” and “password” fields value in accordance to the values stated in the feature file. Finally, the “login” button is clicked. The then part contains the two assertions defined for this test script. The first verifies whether the Overview screen is shown and the second checks that the appointment table is shown on the screen.

```
def "ValidLogin"() {
    given:
        to (org.specification.calendar.LoginScreen);
    when:
        username.value "johnDoe"
    and:
        password.value "Password"
    and:
        login.click
    then:
        at OverviewScreen
    and:
        waitFor { appointments }
}
```

Listing 6: Generated Feature File.

5 EXPERIMENT

An AB/BA crossover experiment was designed and conducted to verify the efficiency of specifying automatically executable GUI test cases with Slang in comparison to JBehave.

5.1 Method

Participants were recruited by email from a Software Engineering course. Approximately one hundred students were contacted. Seventeen students volunteered to participate in the two hour experiment. Participants ranged in age from nineteen to thirty with a mean age of twenty-two. Eleven participants studied information systems, three participants studied computer science, two studied mathematics and one participant studied education. All participants had attended at least two study modules that included programming. A majority of 59 percent writes computer programs as part of study modules. Except for one participant, all volunteers had heard about automated testing and 70 percent had used automated testing at least once. The participants were allocated at random to two experimental groups with eight and nine participants, respectively.

Before the experiment started participants completed a self-assessment questionnaire. Information was sought in relation to age, number of semesters completed and program of study. Further questions related to programming experience and self-rated programming experience in comparison to peers. Participants were asked to indicate the number of modules taken at university that

included writing computer programs as well as occasions in which participants had implemented computer programs such as open source projects or as part of student work experience. Final questions asked about experience in automated testing and automated graphical user interface testing.

Following the completion of each experimental period, a tool-specific questionnaire was administered to all participants. The questionnaire used a five item Likert scale for the first four questions. The fifth question provided “yes”, “no”, “don’t know” as possible responses. The first question asked information about how well the experimental tasks were described. Afterwards, participants were asked to indicate how comfortably they felt specifying feature and mapping files using Slang or story files and test method implementations using JBehave. Further, the questionnaires sought information in relation to the tool support offered by Slang or JBehave. The fifth question asked participants if they would recommend Slang or JBehave to a friend.

Two hours in total were scheduled for questionnaire, training and the experiment. The first ten minute block of the training session started with explaining the experimental setting. Afterward, participants were introduced to the basic concepts of behavior-driven design. Within the next twenty minutes the JBehave tooling was presented. Besides the theoretical explanation, participants were asked to complete a sample task structured exactly as the experimental tasks using JBehave. Participants were handed a list of available commands and assertions per widgets as additional material to ease test case creation. Subsequently, a sample solution was discussed with the participants. The second twenty minutes training block was structured exactly the same, yet, introducing Slang. Participants were encouraged to ask questions during the training period in order to be well prepared for the experimental session in which no questions were allowed.

		Program Period	Calendar		Movie	
			P1		P2	
Group	Sequence	Technique	J	S	J	S
Group I:	JS		X			X
Group II:	SJ			X	X	

Table 1: Experimental Design.

The experiment was scheduled for the second hour which was divided in two thirty minute periods consisting of five minutes preparation, twenty minutes experiment, and five minutes evaluation. Within the five minutes preparation time of each experimental period the task descriptions were handed out to the participants. The task description explained which test cases should be specified for the different programs. The task description also explained when to save the files that were altered during the experiment. During the first experimental period test cases for the Calendar application that support creating, deleting, editing and viewing the details of an appointment had to be specified using Slang or JBehave. In the

second period a Movie database that can be searched for movies had to be considered. The editing of details had to be tested using Slang or JBehave.

Table 1 shows that in the experimental design program and period are confounded. To avoid measuring errors, the participants started the Eclipse IDE on their computers for Slang or JBehave before the experimental period started. In addition to the task descriptions, the printed Wireframesketcher GUI mockups were given to the participants. Further, participants were allowed to use the list of available commands and assertions for each widget from the training session.

Each participant had 20 minutes to specify a maximum of 4 automated test cases. The time for each test case was measured and used as basis for the analysis of efficiency and technical merit of Slang and JBehave. Since both Slang and JBehave are file based it was possible to reconstruct the time taken to complete each test case based on the file history stored by the Eclipse IDE. After the experiment the participant files created during the experiment and their history were collected. The time per test case was accumulated per participant and divided by the number of fully and correctly specified test cases to compute the average time per test case. Figure 5 shows the individual results for each participant. The experiment was executed as an off-line project. Participants were instructed to work independently and to not discuss the solution with their peers.

The integrated development environment Eclipse was used to specify the test cases with Slang or JBehave. Eclipse keeps a file history that is updated every time a file is saved. All materials and results including the questionnaires, task descriptions, Eclipse runtimes, and the raw result data are available for review online at <https://tinyurl.com/SlangMaterialAndResults>.

A two period/two treatment crossover design was chosen as documented by Milliken [20]. The experiment tests the hypothesis that the efficiency of Slang and JBehave is equal. If this hypothesis can be statistically rejected (as it will), one approach (namely Slang) will turn out to be more efficient.

As presented in Table 1 the two groups adhere to a different sequence of applying either Slang or JBehave first. Group 1 applied JBehave in the first period followed by Slang in period two, while Group 2 applied Slang in period one followed by JBehave in period two. It is hypothesized that the sequence does not affect the measurement. Since both Slang and JBehave were introduced before the experiment and participants had the opportunity to gain practice in both tools the carryover effect should be minimal. Nevertheless, a factor for carryover was included in the design. As stated by Vegas et al [27] three variables are unavoidably confounded in AB/BA crossover designs, namely sequence, carryover, period*treatment interaction. In the present study sequence as a factor was included in the model as discussed in Section 5.2. To control error of measurement participants were allocated alternate workstations according to sequence allocation.

The experiment was designed to compare the efficiency of specifying automatically executable test cases with Slang to the efficiency of existing tools for defining automatically executable BDD-like test cases, such as JBehave [15], Cucumber [35], or Squish [29]. Therefore, the comparison to alternative approaches for defining graphical user interface test cases such as Sikuli [26] or Silenium

[25] as well as model-based testing approaches such as Guitar [21] are beyond the scope of this paper. All three BDD tools mentioned above support the gherkin language, provide sophisticated editor support as Eclipse IDE plugins, and generate test methods for implementing the scenario descriptions. Since it would have been difficult to introduce all three tools within a reasonable time frame, only one tool was chosen as representative for BDD-like test case specification. Due to the fact that Java was the only programming language all participants mastered, Squish that does not support test scripts written in Java was excluded. Although JBehave and Cucumber are equivalently suitable based on their features, JBehave was chosen as tool of comparison because of the mature Eclipse IDE integration. This is particularly evident in the code completion offered by the JBehave editor that provides not only potential keywords, but also sample templates to quickly create specifications.

Data was analyzed using a linear mixed model that is recommended for repeated measures in a within-subject crossover design [27]. The data was analyzed using R [22] with the test in linear mixed models package [16] that enhances the lme4 package [2] by tests such as type III F test for fixed effects.

5.2 Results

The descriptive statistics of the experiment are shown in Table 2. As presented by Table 2 it took 174 seconds on average to specify an automated test case using Slang. In comparison, using JBehave 476 second were needed on average. The significance threshold was set at 0.05.

Technique	Mean	Std. Dev.	Std. Error	N	Sig.
JBehave	476	174	42	17	<.001
Slang	174	69	17	17	<.001
Total	325	121		34	

Table 2: Required Time for Creating Automated Test Cases (in seconds).

Figure 5 presents the individual results sorted by sequence. The horizontal axis shows the participant index and the vertical axis shows the time it took on average to fully and correctly specify a test case. The digram area is divided by a vertical line separating the participants that used JBehave before Slang on the left from the participants that used Slang before JBehave on the right. The points on the diagram mark the individual results per participant and technique. Figure 5 indicates that each participant regardless of the sequence in which the techniques were used achieved a more efficient result using Slang.

The linear mixed model used for the analysis includes the following fixed factors: technique (treatment), period (confounded with program), and sequence (confounded with period*technique and carryover). Participant is modeled as random factor nested within sequence.

Analyzing the data using a linear mixed model is based on the assumption that the residuals are distributed normally. In order to apply the linear mixed model, the measured data was transformed using the Box-Cox transformation [11]. The boxcoxmix package

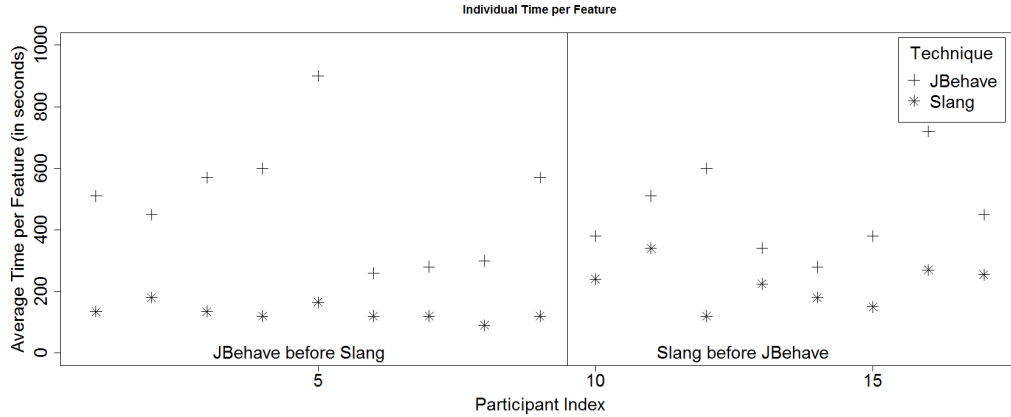


Figure 5: Individual Results Grouped by Sequence.

by Almohaimeed and Einbeck [1] was used to find the optimal λ (0.5) for the transformation.

Table 3 gives the result of the type III F test revealing that technique is the only significant factor (sig. 1.164×10^{-6}). In addition, the test of the remaining fixed factors as reported in Table 2 shows:

- The period factor is not significant and therefore has no effect on the results.
- Sequence is not significant as shown by Table 2.

Based on the results reported in Table 3 the null-hypothesis can be rejected.

Source	Nominator df	Denominator df	F	Sig.
Period	1	14.916	2.906	0.1
Technique	1	15.825	57.646	<.001
Sequence	1	22.358	0.004	0.95

Table 3: Type III Test of Fixed Effects.

The results from the tool-specific questionnaires handed to the participants after each period are shown in Figure 6. The first stacked bar graph shows that 88 percent of participants considered the experimental tasks to be explained well. The second stacked bar graph presents how comfortably participants felt when creating feature or story files using Slang or JBehave, respectively. For Slang the majority of participants stated that they felt very comfortably. For JBehave the majority answered “neutral”. The third stacked bar graphs represents the result for the question how comfortably participants felt when creating mapping files or test method implementations using Slang or JBehave, respectively. While four participants felt uncomfortably using JBehave, all participants using Slang felt at least neutral with a majority feeling “very comfortable”. The fourth stacked bar graph shows the evaluation results for the tool support offered by Slang and JBehave. While 88 percent totally agreed with the statement that they felt well supported by

Slang, only one participant felt well supported by JBehave. Moreover, nearly 60 percent felt not well supported (neutral, disagree, or totally disagree) by the JBehave tooling. The fifth stacked bar graph indicates that all participants will recommend Slang to a friend. Yet, only 20 percent of participants would recommend JBehave.

6 DISCUSSION

The results described in Section 5.2 show that Slang is more efficient than JBehave and works well for users with different backgrounds. Additionally, the quantitative results were endorsed by the qualitative results where participants expressed that they felt comfortably using Slang.

The results from Table 3 show that technique is the only significant factor for the increased performance so that the null hypothesis can be rejected. Specifying fully automated test cases using Slang on average needs 63% less time than using JBehave. The main advantage of Slang is the ability to fully generate the test methods required to execute the scenarios. Leveraging, the higher degree of automation for making test cases executable is the basis for the increased efficiency.

In addition, as shown by Figure 6 participants have reported that they felt better supported by Slang. This can be explained by the mature editor support and the context sensitive integration of wireframe models that guides participants through the process of creating scenarios and mappings.

The experiment has shown that the integration of wireframe models and scenario descriptions enables the complete generation of test methods. As Lazăr et al. [14] have shown how scenario descriptions can be combined with UML artifacts to generate test methods for algorithms, we have shown that scenario descriptions combined with wireframe models can be used to generate test methods for graphical user interfaces.

Regarding one of the main disadvantages of automated graphical user interface tests, namely being expensive to write, the experimental results show that Slang decreases the time required per test case. Our approach, automates the most time consuming part of the test case creation, which is the implementation of the page objects. Thereby, Slang is more efficient than JBehave and as we

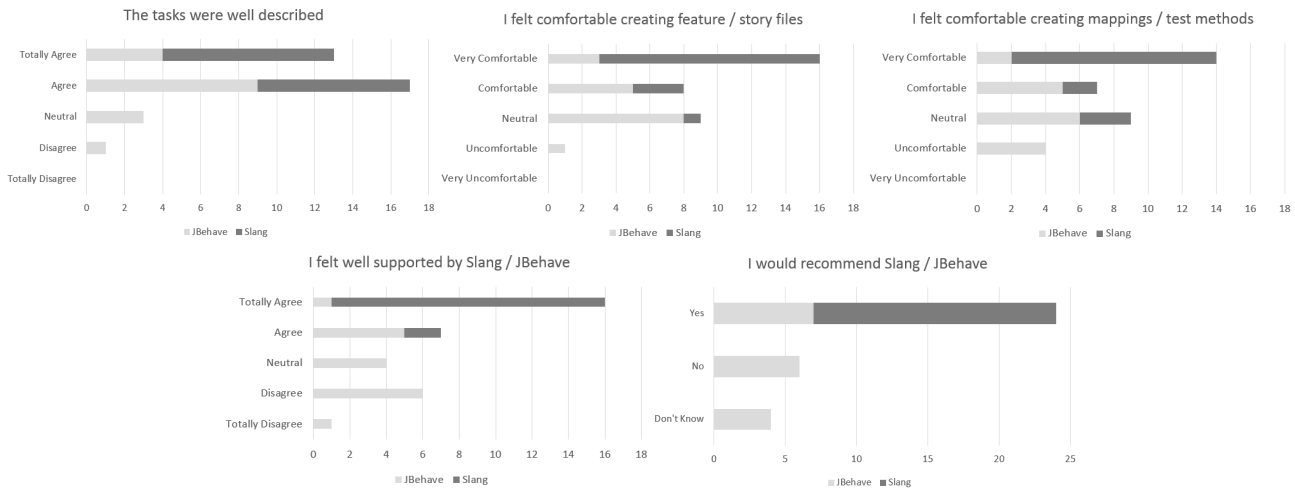


Figure 6: Participant Evaluation Results of Slang and JBehave.

believe more efficient than other behavior-driven test tools such as Cucumber or Squish since they all require manual test method and page object implementation.

As explained in Section 2 our approach is less flexible and complex than natural language processing but more flexible and readable than keyword driven testing. Since test cases could be specified with Slang as well as with JBehave the additional restrictions introduced by the Feature DSL did not prevent the test case creation. Thereby, Slang allows non-technical users to specify automated test cases. Due to the fact that all participants had at least some programming experience, they were able to create test cases with JBehave as well as with Slang. However, as Figure 5 reveals, creating test cases using JBehave took longer, especially, for participants self-assessing their programming skills as “unexperienced” or “very unexperienced” compared to their peers.

Threats to validity. The validity of the experimental results is a key issue. The AB/BA crossover design shown in Table 1 addresses the inherent threats to validity [27] in the following way:

Period and program are confounded in the selected design. As indicated by Table 3 period is not a significant factor in our model. Therefore, all threats to validity associated with the experimental session, namely learning by practice and tiredness/boredom, are not occurring. Since period and program are confounded, program is also not significant.

There is no threat of technique learning, because participants only apply each technique once. Thereby, any improvement due to repeated application of a technique is minimized. Moreover, there is no threat of object learning since every participant applies each technique only to one requirement specification (Calendar or Movie application). The copy threat can be ruled out, since participants use a different Eclipse installation and workspace for the requirement specification in each period.

As contained in Table 3 sequence is not a significant factor in the model. Therefore, carryover as a threat to validity is not present. Moreover, period $\times$ technique interaction which is also confounded with sequence is not occurring. Hence, there is no optimal sequence

validity threat present. An additional threat is the fact that Slang is only compared to JBehave while there are other tools, such as Cucumber [35] or Squish [29] that may be used for behavior-driven development. However, since all tools do not support complete generation of test methods they are equally suitable for the experiment.

Inference. The experimental results show that Slang offers a language to efficiently specify automated graphical user interface test cases. Hence, even users with limited programming skills and non-technical experts are able to specify test cases in form of readable sentences. Slang is most relevant, effective, and efficient for projects using wireframe models. In a commercial setting Slang would enable team members with various background to participate in creating test cases that are automated eventually by the introduced generators.

Component	Number of test cases	Percentage of Generated Testing Code
SWT1	42	94
SWT2	15	100
SWT3	4	100

Table 4: Number of Test Cases and Percentage of Generated Code per Component.

Table 4 shows the results from applying Slang in three differently sized software components of a larger practical projects in the financial industry using SWT as GUI framework. While the four test cases of SWT3 are mainly concerned with specifying the general layout of the central screen, SWT2 also provides more sophisticated test cases for analyzing, e.g., the page flow. SWT1 contains the most feature specifications that test the creation of particular elements within a table. In contrast to SWT2 and SWT3 that confirm the experimental results, SWT1 required some manual code, e.g., to close an editor in a dirty state (with unsaved changes). The percentage of generated code was determined by analyzing the lines of code (LOC) manually implemented for `ImplementedCommands` and `ImplementedAssertions` in relation to the overall lines of produced

test code. Although LOC are a debatable measure for computer programs, they in this case show that the majority of code can be created automatically. In addition, the feedback from the industry partner confirmed that Slang is easy to use. Especially the editor support and the sophisticated integration with wireframe mockups was emphasized.

7 CONCLUSION

Our behavior-driven GUI testing approach integrates BDD-like feature descriptions with wireframe model elements. By adding a mapping definition, automatically executable GUI test cases can be generated completely. The approach aims to mitigate the three main problems of automated GUI test cases, namely being expensive to create, time consuming to execute, and brittle.

The present study has clearly demonstrated that test cases can be obtained much more efficiently since existing artifacts such as wireframes and feature descriptions are reused. Moreover, the experiment has shown that Slang on average requires 63% less time than existing approaches such as JBehave for specifying automated GUI test cases. In addition, applying Slang in three software development projects in the financial sector has confirmed the experimental findings in terms of at least 96 percent generated code and highly satisfied users that felt well supported by Slang.

Future work may focus on comparing Slang to other behavior-driven development tools, such as Cucumber or Squish. Moreover, additional research is required to verify the expected effects of Slang on mitigating the risk of brittle test cases.

The contribution of this paper is threefold: first, a Specification Language (Slang) has been presented that consists of feature and mapping descriptions. Second, two generators have been developed that combine wireframe, feature and mapping definition to generate automatically executable GUI test cases. Thirdly, the experimental results clearly show the technical merit and improved efficiency of Slang not only in example projects but also in industry context. Furthermore, Slang was positively endorsed by all those participating in the experiments and in the projects in the financial sector.

REFERENCES

- [1] Amani Almohaimed and Jochen Einbeck. 2017. *boxcoxmix: Response Transformations for Random Effect and Variance Component Models*. <https://CRAN.R-project.org/package=boxcoxmix> R package version 0.15.
- [2] Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. 2015. Fitting Linear Mixed-Effects Models Using lme4. *Journal of Statistical Software* 67, 1 (2015), 1–48. <https://doi.org/10.18637/jss.v067.i01>
- [3] Lorenzo Bettini. 2013. *Implementing domain-specific languages with Xtext and Xtend*. Packt Pub, Birmingham, UK.
- [4] Daniel M. Brown. 2011. *Communicating design: Developing web site documentation for design and planning*. - Previous ed.: 2007. - Includes index (2nd ed. ed.). New Riders, Berkeley, Calif.
- [5] C. Solis and X. Wang. 2011. A Study of the Characteristics of Behaviour Driven Development. In *2011 37th EUROMICRO Conference on Software Engineering and Advanced Applications*. IEEE Computer Society, Washington, DC, USA, 383–387. <https://doi.org/10.1109/SEAA.2011.76>
- [6] Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*. ACM, ACM, New York, NY, USA, 160–167.
- [7] Adrien Coyette, Suzanne Kieffer, and Jean Vanderdonck. 2007. *Multi-fidelity Prototyping of User Interfaces*. Springer-Verlag, Berlin, Heidelberg, 150–164. https://doi.org/10.1007/978-3-540-74796-3_16
- [8] Dan North. 2018. Introducing BDD. (2018). <https://dannorth.net/introducing-bdd/>
- [9] Eclipse. 2018. SWT - The Standard Widget Toolkit. (2018). <https://www.eclipse.org/swt/>
- [10] Eric Evans. 2014. *Domain-driven design: Tackling complexity in the heart of software* (20. print ed.). Addison-Wesley, Boston, Mass. and Munich [u.a.].
- [11] George E. P. Box and David R. Cox. 1964. An analysis of transformations. *Journal of the Royal Statistical Society* 26, 2 (1964), 211–252.
- [12] groovy-lang.org. 2018. groovy - A multi-faceted language for the Java platform. (2018). <http://groovy-lang.org/>
- [13] Zahid Hussain, Wolfgang Slany, and Andreas Holzinger. 2009. Current State of Agile User-Centered Design: A Survey. In *HCI and Usability for e-Inclusion: 5th Symposium of the Workgroup Human-Computer Interaction and Usability Engineering of the Austrian Computer Society, USAB 2009, Linz, Austria, November 9-10, 2009 Proceedings*, Andreas Holzinger and Klaus Miesenberger (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 416–427. https://doi.org/10.1007/978-3-642-10308-7_30
- [14] Ioan Lazăr, Simona Motogna, and Bazil Părv. 2010. Behaviour-Driven Development of Foundational UML Components. *Electronic Notes in Theoretical Computer Science* 264, 1 (2010), 91–105. <https://doi.org/10.1016/j.entcs.2010.07.007>
- [15] JBehave. 2018. What is JBehave. (2018). <https://jbehave.org/reference/stable/index.html>
- [16] Alexandra Kuznetsova, Per B. Brockhoff, and Rune H. B. Christensen. 2017. lmerTest Package: Tests in Linear Mixed Effects Models. *Journal of Statistical Software* 82, 13 (2017), 1–26. <https://doi.org/10.18637/jss.v082.i13>
- [17] Esteban Robles Luna, Julián Grigera, and Gustavo Rossi. 2009. Bridging test and model-driven approaches in web engineering. In *International Conference on Web Engineering*. Springer, Springer Berlin Heidelberg, Berlin, Heidelberg, 136–150.
- [18] Martin Fowler. 2012. PageObject. (2012). <https://martinfowler.com/bliki/TestPyramid.html>
- [19] Martin Fowler. 2013. PageObject. (2013). <https://martinfowler.com/bliki/PageObject.html>
- [20] George A. Milliken and Dallas E. Johnson. 2009-. *Analysis of messy data . Volume 1 Designed Experiments/ by George A. Milliken, Dallas E. Johnson* (2nd ed. ed.). CRC Press, Boca Raton.
- [21] Bao N Nguyen, Bryan Robbins, Ishan Banerjee, and Atif Memon. 2014. GUITAR: an innovative tool for automated testing of GUI-driven software. *Automated software engineering* 21, 1 (2014), 65–105.
- [22] R Core Team. 2017. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>
- [23] José Matias Rivero, Gustavo Rossi, Julián Grigera, Juan Burella, Esteban Robles Luna, and Silvia Gordillo. 2010. *From Mockups to User Interface Models: An Extensible Model Driven Approach*. Springer Berlin Heidelberg, Berlin, Heidelberg, 13–24. https://doi.org/10.1007/978-3-642-16985-4_2
- [24] Oscar Sanchez Ramon, Jesus Garcia Molina, Jesus Sanchez Cuadrado, and Jean Vanderdonck. 2013. GUI Generation from Wireframes. (2013). https://dial.uclouvain.be/download/download.php?pid=boreal:153911&datastream=PDF_01
- [25] Selenium. 2017. SeleniumHQ - Browser Automation. (2017). <https://www.seleniumhq.org/>
- [26] Sikuli. 2017. Sikuli - Automate anything you see. (2017). <http://www.sikuli.org/>
- [27] Sira Vegas, Cecilia Apa, and Natalia Juristo. 2016. Crossover Designs in Software Engineering Experiments: Benefits and Perils. *IEEE Transactions on Software Engineering* 42, 2 (2016), 120–135.
- [28] Mathias Soeken, Robert Wille, and Rolf Drechsler. 2012. Assisted Behavior Driven Development Using Natural Language Processing. In *Proceedings of the 50th International Conference on Objects, Models, Components, Patterns (TOOLS'12)*. Springer-Verlag, Berlin, Heidelberg, 269–287. https://doi.org/10.1007/978-3-642-30561-0_19
- [29] Squish. 2017. Squish - Reliable GUI Test Automation that works. (2017). <https://www.froglogic.com/squish/>
- [30] Mark Utting and Bruno Legeard. 2007. *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- [31] Lars Vogel. 2015. *Eclipse rich client platform: The complete guide to Eclipse application development* (third ed.). Vogella, Hamburg.
- [32] Markus Völter and Thomas Stahl. 2013. *Model-Driven Software Development: Technology, Engineering, Management*. John Wiley & Sons, New York, NY.
- [33] W3C. 2018. Document Object Model (DOM). (2018). <https://www.w3.org/DOM/>
- [34] Wireframesketcher. 2018. Wireframesketcher. (2018). <http://www.wireframesketcher.com/>
- [35] Matt Wynne, Aslak Hellesøy, Steve Tooke, and Jacquelyn Carter. 2017. *The Cucumber book: Behaviour-driven development for testers and developers* (second ed.). The Pragmatic Bookshelf, trademark of The Pragmatic Programmers, LLC, Raleigh, North Carolina.
- [36] Xtend. 2018. Xtend Introduction. (2018). <https://eclipse.org/xtend/documentation/>
- [37] Ayal Zylberman and Aviram Shotten. 2010. Test language: Introduction to keyword driven testing. (2010). <http://www.methodsandtools.com/archive/archive.php?id=108>